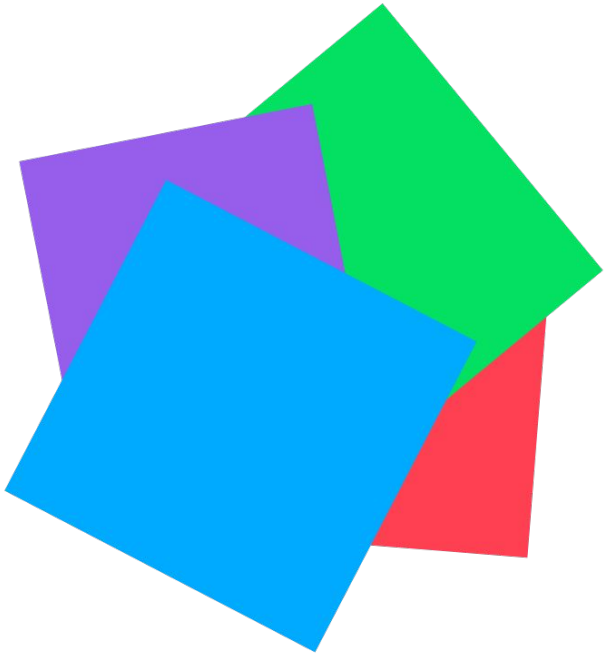
Паттерны программирования



Avito Start

Результаты обучения

1. Моделировать состояние объектов

2. Комбинировать бизнес правила без дублирования

3. Проверять инварианты, ошибки и стандартные контракты Go



Паттерны и идиомы Go в пиццерии

01 **Состояния и действия**

State ограничивает запуск приготовления. Command представляет отмену заказа.

02 **Решения и правила**

Цепочка выбирает маршрут заказа. Specification проверяет условия доставки.

03 **Анализ и настройки**

Visitor печатает правила приёма. Functional Options настраивает кухню.

Сценарий: заказ на кухне

01 **Жизненный цикл**

Принятый заказ готовится, ожидает выдачи и передаётся гостю.

02 **Правила приёма**

Проверяем наличие пицц, размер заказа и адрес доставки. Решение должно иметь причину.

03 **Изменения**

Появляются новые действия кухни и правила маршрутизации. Расширяем подтверждённые точки изменения.

State: жизненный цикл заказа

01 **accepted**

StartBake переводит в baking. Cancel переводит в cancelled.

02 **baking и ready**

MarkReady переводит baking в ready. HandOut переводит ready в completed.

03 **completed и cancelled**

Терминальные состояния. Повторный запуск печи запрещён.

State: запуск приготовления

```
type State interface {
    StartBake(*Order) error
}
type Accepted struct{}
type Baking struct{}

func (Accepted) StartBake(o *Order) error {
    o.state = Baking{}
    return nil
}
func (Baking) StartBake(*Order) error {
    return ErrTransition
}
```

Order делегирует StartBake своему текущему состоянию

State: печь нельзя запустить дважды

```
func TestStartBakeTwice(t *testing.T) {
    o := NewOrder() // state = Accepted{}
    if err := o.StartBake(); err != nil {
        t.Fatal(err)
    }
    err := o.StartBake()
    if !errors.Is(err, ErrTransition) {
        t.Fatalf("expected transition error: %v", err)
    }
}
```

Второй вызов не меняет состояние baking

Command: отмена как действие

01 **Задача**

Касса и приложение инициируют отмену заказа с одинаковыми бизнес-правилами.

02 **Решение**

CancelCommand хранит номер заказа. Исполнитель получает зависимость Orders.

03 **Граница**

Команда выражает намерение. Проверку допустимости отмены выполняет сервис заказа.

Command для отмены заказа

```
type Command interface {
    Execute(context.Context) error
}
type CancelCommand struct {
    id int64
    orders Orders
}
func (c CancelCommand) Execute(ctx context.Context) error {
    return c.orders.Cancel(ctx, c.id)
}
```

Касса вызывает cmd.Execute(ctx)

Command: побочные эффекты отмены

01 **Повторное выполнение**

Два нажатия «Отменить» не должны приводить к двум возвратам оплаты. Нужен явный контракт повтора.

02 **Отмена ожидания**

context может прекратить ожидание ответа. Уже выполненный возврат он не откатывает.

03 **Обратная операция**

Отмена заказа не возвращает испечённую пиццу в исходное состояние. Компенсация требует своих правил.

Chain of Responsibility: маршрут заказа

01 **Задача**

Недоступная пицца означает отказ. Большой заказ требует звонка. Остальные заказы идут на кухню.

02 **Решение**

Обработчик возвращает решение или передаёт запрос дальше. Порядок задаёт приоритет.

03 **Контракт**

Первое решение останавливает цепочку. Ошибка зависимости прерывает обработку.

Цепочка правил заказа

```
type Rule func(Order) (Decision, bool, error)

func Decide(o Order, rules []Rule) (Decision, error) {
    for _, rule := range rules {
        decision, handled, err := rule(o)
        if err != nil {
            return Decision{}, err
        }
        if handled {
            return decision, nil
        }
    }
    return Decision{}, ErrNoDecision
}
```

Отказ по меню имеет приоритет над ручной проверкой

Specification: допуск к доставке

01 **Задача**

Адрес и положительная сумма нужны и в предварительной проверке, и при оформлении.

02 **Решение**

Чистые предикаты hasAddress и positive комбинируются через AND.

03 **Цена**

bool не объясняет, почему доставка недоступна. Причины отказа требуют отдельной модели.

Specification для доставки

```
type Spec func(Order) bool

func And(a, b Spec) Spec {
    return func(o Order) bool {
        return a(o) && b(o)
    }
}

hasAddress := Spec(func(o Order) bool {
    return strings.TrimSpace(o.Address) != ""
})
positive := Spec(func(o Order) bool { return o.Total > 0 })
canDeliver := And(hasAddress, positive)
```

canDeliver(order) проверяет заказ без его изменения

Цепочка и спецификация

01 **Цепочка**

Что делать с заказом? Отказать, уточнить детали или передать кухне.

02 **Спецификация**

Подходит ли заказ для доставки? Есть адрес и положительная сумма.

03 **Совместное применение**

Правило цепочки использует canDeliver и возвращает решение с понятной причиной.

Пицца недоступна, а заказ большой. Какое правило цепочки должно сработать первым?

Visitor: операции над правилами

01 **Задача**

У пиццерии есть устойчивые виды правил. К ним добавляются печать условий и сбор статистики.

02 **Решение**

Каждый вид правила вызывает соответствующий метод посетителя.

03 **Цена**

Новый вид правила потребует изменения всех посетителей. Нужна стабильная структура модели.

Visitor: виды правил пиццерии

```
type RuleNode interface { Accept(Visitor) }
type TotalRule struct { Min int }
type AddressRule struct { Required bool }
type Visitor interface {
    VisitTotal(TotalRule)
    VisitAddress(AddressRule)
}
func (r TotalRule) Accept(v Visitor) { v.VisitTotal(r) }
func (r AddressRule) Accept(v Visitor) { v.VisitAddress(r) }
```

Узел выбирает метод для своего вида условия

Visitor: печать условий заказа

```
type Printer struct { Lines []string }

func (p *Printer) VisitTotal(r TotalRule) {
    p.Lines = append(p.Lines,
        fmt.Sprintf("total >= %d kop", r.Min))
}
func (p *Printer) VisitAddress(r AddressRule) {
    p.Lines = append(p.Lines,
        fmt.Sprintf("address required = %t", r.Required))
}

p := &Printer{}
TotalRule{Min: 50_000}.Accept(p)
```

Новая операция добавляет посетителя, сохраняя виды правил

Visitor: цена расширения

01 **Добавился отчёт**

Новый посетитель собирает статистику по условиям меню. TotalRule и AddressRule остаются прежними.

02 **Добавилось правило размера**

Нужен SizeRule и новый метод во всех посетителях, включая Printer.

03 **Маленькая модель**

Если отчёт один, прямой type switch по правилам пиццерии может быть понятнее.

Что придётся изменить, если добавить новый вид правила в модель с Visitor?

Functional Options: настройки кухни

01 **Задача**

Кухне обязательно нужно хранилище заказов. Максимум пицц в заказе является настройкой.

02 **Решение**

Хранилище передаём явно. WithMaxPizzas меняет конфигурацию перед созданием кухни.

03 **Контракт**

По умолчанию 20 пицц. Повтор опции использует последнее значение. Недопустимую настройку отклоняем.

Functional Options: размер заказа

```
type config struct { maxPizzas int }
type Option func(*config) error

func WithMaxPizzas(n int) Option {
    return func(c *config) error {
        if n <= 0 {
            return errors.New("max pizzas must be positive")
        }
        c.maxPizzas = n
        return nil
    }
}
```

Настройка ограничивает число пицц в одном заказе

Avito Start

Functional Options: создание кухни

```
func NewKitchen(repo OrderRepo, opts ...Option) (*Kitchen, error) {
    if repo == nil { return nil, errors.New("nil repo") }
    c := config{maxPizzas: 20}
    for _, opt := range opts {
        if opt == nil { return nil, errors.New("nil option") }
        if err := opt(&c); err != nil { return nil, err }
    }
    return &Kitchen{repo: repo, maxPizzas: c.maxPizzas}, nil
}

kitchen, err := NewKitchen(repo, WithMaxPizzas(10))
```

WithMaxPizzas(10) меняет максимум с 20 на 10

Сборка решения пиццерии

01 **Действия кухни**

Command запускает отмену заказа. State ограничивает отмену и начало приготовления.

02 **Приём заказа**

Цепочка задаёт маршрут. Specification проверяет условия доставки.

03 **Настройка и анализ**

Functional Options задаёт максимум пицц. Visitor печатает описания правил.

Ошибки и результаты заказа

01 **Результат предметной области**

Пиццы нет в меню: Decision{Route: "rejected"} с причиной. Это бизнес-решение.

02 **Нарушение перехода**

Запуск печи второй раз возвращает ErrTransition. Его можно проверить через errors.Is.

03 **Техническая ошибка**

Сбой сохранения заказа прерывает операцию. fmt.Errorf с %w сохраняет исходную причину.

Iterator: iter.Seq в Go 1.23+

01 **Задача**

Обходить только готовые заказы, скрывая способ хранения коллекции.

02 **Контракт**

iter.Seq[Order] передаёт значения через yield. После false обход обязан закончиться.

03 **Граница**

Для одного обхода обычный цикл проще. Ошибки и освобождение внешнего ресурса требуют своего контракта.

Iterator для готовых заказов

```
func ReadyOrders(all []Order) iter.Seq[Order] {
    return func(yield func(Order) bool) {
        for _, o := range all {
            if o.Status() == "ready" {
                if !yield(o) { return }
            }
        }
    }
}
```

Результат ленивый и не создаёт второй срез

Ранняя остановка Iterator

```
for o := range ReadyOrders(orders) {
    fmt.Println("К выдаче:", o.ID)
    break
}
```

После break функция yield возвращает false

Композиция ошибок как контракт Go

01 **Причина**

fmt.Errorf с %w сохраняет связь с исходной ошибкой. errors.Is проверяет известную причину.

02 **Данные**

errors.As получает типизированную ошибку, например дефицит ингредиента.

03 **Граница**

Открытая причина становится частью API. Сравнение строк сообщения ненадёжно.

Типизированная причина отказа кухни

```
type StockError struct { Ingredient string }
func (e *StockError) Error() string {
    return "unavailable: " + e.Ingredient
}

err := fmt.Errorf("order %d: %w", order.ID,
    &StockError{Ingredient: "cheese"})
var stock *StockError
if errors.As(err, &stock) {
    log.Print("Нет ингредиента: ", stock.Ingredient)
}
```

Причина остаётся доступной после оборачивания

Несколько причин: errors.Join

```
func ValidateDelivery(address string, count int) error {
    var problems []error
    if strings.TrimSpace(address) == "" {
        problems = append(problems, ErrAddress)
    }
    if count <= 0 {
        problems = append(problems, ErrEmptyOrder)
    }
    return errors.Join(problems...)
}
```

errors.Is находит каждую из сохранённых причин

Composite: товары и вложенные комбо

01 **Задача**

В заказе есть отдельные товары и комбо. Комбо может содержать другие комбо.

02 **Общий интерфейс**

Product и Combo реализуют Item с методом Price(). Код заказа работает с любым Item.

03 **Рекурсивный состав**

Combo хранит []Item и складывает цены элементов. Вложенное комбо считает свою цену тем же способом.

Composite: общий интерфейс цены

```
type Item interface { Price() int }
type Product struct { Kop int }
func (p Product) Price() int { return p.Kop }

type Combo struct { Items []Item }
func (c Combo) Price() int {
    total := 0
    for _, item := range c.Items {
        total += item.Price()
    }
    return total
}
```

Product возвращает свою цену. Combo суммирует цены вложенных Item

Composite: комбо внутри комбо

```
pizza := Product{Kop: 60_000}
drink := Product{Kop: 10_000}
dessert := Product{Kop: 15_000}
lunch := Combo{Items: []Item{pizza, drink}}

party := Combo{Items: []Item{
    lunch, lunch, dessert,
}}
var order Item = party
fmt.Println(order.Price()) // 155000 копеек
```

Два обеденных комбо и десерт стоят 1 550 ₽. Все цены в коде в копейках

В комбо добавили ещё одно комбо. Почему код подсчёта цены не нужно менять?

CQRS: запуск печи и табло кухни

01 **Разные задачи**

Запуск печи меняет заказ и проверяет переход состояния. Табло показывает готовые заказы.

02 **Разные модели**

Команда работает с доменным Order. Запрос возвращает KitchenRow с полями для табло.

03 **Разделение ответственности**

Обработчик команды меняет данные. Обработчик запроса только читает их. БД может быть общей.

CQRS: команда запуска печи

```
type StartBake struct { OrderID int64 }

func (h BakeHandler) Handle(ctx context.Context, cmd StartBake) error {
    order, err := h.orders.Get(ctx, cmd.OrderID)
    if err != nil { return err }
    if err := order.StartBake(); err != nil { return err }
    return h.orders.Save(ctx, order)
}
```

Команда проверяет переход через Order. Сохранение выполняется после проверки

CQRS: запрос для табло кухни

```
type KitchenRow struct {
    ID int64
    PizzaCount int
}
type KitchenQueries interface {
    Ready(context.Context) ([]KitchenRow, error)
}
const readySQL = `
SELECT id, pizza_count
FROM orders
WHERE status = 'ready'
ORDER BY id`
```

Запрос возвращает DTO для табло. Доменный Order загружать не требуется

CQRS: польза и цена разделения

01 **Начальный вариант**

Один сервис и одна БД. Разные модели для записи и чтения. Очереди и Event Sourcing необязательны.

02 **Польза и цена**

Правила записи и формат табло развиваются отдельно. Для простого CRUD разделение может быть лишним.

03 **Отдельная БД чтения**

Асинхронное обновление табло даёт задержку. Потребуются доставка изменений и обработка повторов.

Антипаттерны в модели пиццерии

01 **God Object под новым именем**

KitchenEngine знает меню, платёжный SDK, печь и все правила доставки.

02 **Spaghetti из обработчиков**

Регистрация скрывает приоритет отказа и момент запуска печи. Шаги и остановка должны быть явными.

03 **Premature Abstraction**

DSL ресторана ради одной проверки адреса. Начинаем с конкретного правила.

Проверки после расширения

01 **Переходы и приоритет**

Второй StartBake сохраняет baking. Недоступная пицца останавливает цепочку раньше review.

02 **Чистота правил**

canDeliver не меняет Order. Новый Visitor сохраняет вывод старого Printer.

03 **Контракт кухни**

WithMaxPizzas(0) не создаёт Kitchen. Ошибка хранилища доступна через errors.Is.

Итоги занятия

01 State задаёт переходы. CQRS разделяет запись и чтение

02 Iterator соблюдает контракт ранней остановки

03 Ошибки сохраняют причины и типы

04 Composite считает цену товаров и вложенных комбо

05 **context передаёт срок и отмену по цепочке вызовов**

Материалы для продолжения

01 **Классические идеи**

GoF: State, Command, Chain of Responsibility, Visitor, Composite

02 **Правила и архитектура**

Evans / Fowler: Specifications и CQRS
Dave Cheney: Functional options for friendly APIs

03 **Итераторы и ошибки Go**

iter.Seq, errors и стандартные контракты Go


Q&A
time